

849. Maximize Distance to Closest Person

<https://leetcode.com/problems/maximize-distance-to-closest-person/description/>

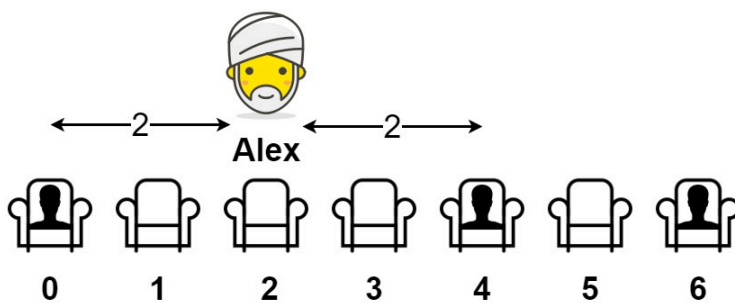
Дата: 16.01.2026

Автор: Андрей Трушин, Глеб Михайлов

Эксклюзивно для курса "Алгоритмы с Глебом Михайловым" 🐯

Условие задачи:

Описание задачи:



Интроверт (социофоб) идет в кинотеатр. У нас есть ряд мест, представленный массивом `seats`, где 1 означает занятое место, а 0 — свободное. Нужно найти место для Алекса так, чтобы расстояние до ближайшего человека было максимальным.

Задача "Maximize Distance to Closest Person" часто встречается на собеседованиях в Яндекс и VK. Это задача уровня Medium на LeetCode.

Дано:

- массив `seats` (индекс начинается с 0), состоящий из 0 и 1:
 - 1 — место занято человеком
 - 0 — свободное место

Нужно:

- Найти максимально возможное расстояние до ближайшего занятого места

Примеры:

- **Пример 1**

Входные данные: `[1, 0, 0, 0, 1, 0, 1]`

Ответ: 2

Объяснение: Если сесть на место с индексом 2, расстояние до ближайшего человека будет 2 (два шага).

- **Пример 2**

Входные данные: [0, 1]

Ответ: 1

Объяснение: Единственное свободное место — индекс 0, ближайший человек на индексе 1, расстояние = 1.

- **Пример 3**

Входные данные: [0, 0, 0, 1]

Ответ: 3

Объяснение: Если сесть на место 0 (самое левое), расстояние будет 3 шага.

Решение:

Декомпозиция задачи

Ключ к решению — разбить задачу на три независимые подзадачи:

1. Посадка слева — пытаемся посадить челика на самое левое место (начало массива)
2. Посадка справа — пытаемся посадить на самое правое место (конец массива)
3. Посадка посередине — пытаемся посадить между двумя занятыми местами

После решения всех трёх подзадач выбираем максимальное расстояние из трёх вариантов.

Подзадача 1: Посадка слева

Если слева есть свободные места перед первым человеком, расстояние равно количеству нулей с начала массива

Алгоритм:

```
# left -- количество нулей в начале
left = 0
while seats[left] == 0:
    left += 1
```

Переменная `left` будет равна индексу первого занятого места, что и есть расстояние до него.

Пример:

`[0, 0, 0, 1] → left = 3`

`[1, 0, 0, 0] → left = 0` (цикл `while` не запустится)

Подзадача 2: Посадка справа

Аналогично посадке слева, но идём с конца массива.

Алгоритм:

```
# right -- количество нулей в конце
right = 0
right_ind = len(seats) - 1
while seats[right_ind] == 0:
    right += 1
    right_ind -= 1
```

Переменная `right` – кол-во незанятых мест справа, что и есть расстояние до первого занятого места. `right_ind` отслеживаем для третьей подзадачи.

Пример:

`[0, 1, 0, 0, 0] → right = 3`

Подзадача 3: Посадка посередине

Находим максимальную последовательность нулей между двумя занятыми местами (единицами). Оптимально сесть в середину такой последовательности. Для расчета кол-ва мест будем использовать счётчик `mid`. При этом таких последовательностей может быть несколько, поэтому необходимо отобрать максимальную из них `mid_max`. После этого необходимо рассчитать место посередине – это и будет максимальной дистанцией до ближайшего занятого места.

Алгоритм:

```
mid = 0 # Счетчик свободных мест
mid_max = 0

for i in range(left, right_ind + 1):
    if seats[i] == 0:
        mid += 1
        mid_max = max(mid_max, mid)
    else:
```

```
mid = 0

mid_max = (mid_max + 1) // 2
```

Важный трюк: формула $(mid_max + 1) // 2$ работает для обоих случаев:

- Нечётное количество свободных мест (нулей): $(3 + 1) // 2 = 2$
- Чётное количество свободных мест (нулей): $(4 + 1) // 2 = 2$

Пример:

```
[1, 0, 0, 0, 1] → mid_max = 3 → (3+1)//2 = 2
[1, 0, 0, 0, 0, 1] → mid_max = 4 → (4+1)//2 = 2
```

Код на Python

```
from typing import List

class Solution:
    def maxDistToClosest(self, seats: List[int]) -> int:

        # left -- количество нулей в начале
        left = 0
        while seats[left] == 0:
            left += 1

        # right -- количество нулей в конце
        right = 0
        right_ind = len(seats) - 1
        while seats[right_ind] == 0:
            right += 1
            right_ind -= 1

        # mid -- ищем только между первой и последней единицей
        mid = 0
        mid_max = 0

        for i in range(left, right_ind + 1):
            if seats[i] == 0:
                mid += 1
                mid_max = max(mid_max, mid)
            else:
                mid = 0

        mid_max = (mid_max + 1) // 2

        return max(left, right, mid_max)
```

Итоговая сложность алгоритма:

Время:

$O(n)$ — в общем случае переберём все элементы массива один раз. Сопутствующие операции выполняются за константное время

Память:

$O(1)$ — Используются несколько дополнительных переменных для хранения индексов и расстояний

Follow-up: вернуть индекс максимально удалённого места

Если нужно вернуть не только максимальную дистанцию, но и индекс посадочного места, можно использовать ту же декомпозицию на три подзадачи. В каждой подзадаче будем формировать кортеж (дистанция, индекс), а в финале — выбирать максимум по этим кортежам. Python сравнивает кортежи лексикографически: сначала по дистанции, и индекс «приедет» вместе с лучшим кандидатом.

Идея по индексам:

Подзадача 1 (слева): дистанция = left, индекс = 0 — садимся в самый левый край.

Подзадача 2 (справа): дистанция = right, индекс = $n - 1$ — садимся в самый правый край.

Подзадача 3 (середина): при проходе запоминаем правую границу самой длинной серии нулей mid_end. Левая граница серии = mid_end - mid_max + 1. Дистанция = (mid_max + 1) // 2. Индекс посадки = mid_end - mid_max + mid_dist (это эквивалентно «левый край серии + (mid_max - 1) // 2»).

Код на Python:

```
class Solution:
    def maxDistToClosestWithIndex(self, seats: List[int]) -> Tuple[int,
int]:
        n = len(seats)

        # Подзадача 1: посадка слева
        left = 0
        while seats[left] == 0:
```

```

        left += 1
    left_result = (left, 0)

    # Подзадача 2: посадка справа
    right = 0
    right_ind = n - 1
    while seats[right_ind] == 0:
        right += 1
        right_ind -= 1
    right_result = (right, n - 1)

    # Подзадача 3: посадка посередине
    mid = 0
    mid_max = 0
    mid_end = -1 # правый индекс лучшей серии нулей
    for i in range(left, right_ind + 1):
        if seats[i] == 0:
            mid += 1
            if mid > mid_max:
                mid_max = mid
                mid_end = i
        else:
            mid = 0

    mid_dist = (mid_max + 1) // 2
    mid_index = mid_end - mid_max + mid_dist if mid_max > 0 else -1
    mid_result = (mid_dist, mid_index)

    # max по кортежам сравнит сначала дистанции и вернёт индекс
    вместе с ней
    return max(left_result, right_result, mid_result)

```

Проверка на примерах:

[1, 0, 0, 0, 1, 0, 1] → left=(0,0), right=(0,6), mid: серия из 3 нулей, mid_end=3, mid_dist=2, mid_index = 3 - 3 + 2 = 2. Ответ: (2, 2).

[1, 0, 0, 0] → left=(0,0), right=(3,3), mid не запускается (right_ind=0). Ответ: (3, 3).

[0, 0, 1] → left=(2,0), right=(0,2). Ответ: (2, 0).

[1, 0, 0, 0, 0, 1] → mid_max=4, mid_end=4, mid_dist=2, mid_index = 4 - 4 + 2 = 2. Ответ: (2, 2) — индекс 2 как левая середина из {2, 3}.



